RELAZIONE SISTEMI OPERATIVI

Cognome: **ZEUKENG TSAFACK**

Nome: **MARTIN LUTHER**

Matricola: **VR510379**

Data: 16/07/2025

Indice

1. **Introduzione**
2. **Struttura Generale del Sistema**
    a. 2.1 Il Componente Client – the_client.c
    b. 2.2 Il Componente Server – the_server.c
3. **Analisi Tecnologica del Sistema**
    a. 3.A Meccanismi FIFO e Generazione Dinamica dei Canali di Risposta
    b. 3.B Hashing SHA-256 tramite Libreria OpenSSL
    c. 3.C Politica di Priorità Basata sulla Dimensione dei File
    d. 3.D Architettura Concorrente: Thread, Blocchi Critici e Condizioni
    e. 3.E Strategia di Memorizzazione dei Risultati e Coordinamento delle Elaborazioni
4. **Esame di Sezioni Chiave del Codice Sorgente**
    a. 4.A Costruzione delle Richieste nel Lato Client
    b. 4.B Algoritmo di Hash: Funzione compute_sha256
    c. 4.C Inserimento Ordinato: Funzione enqueue_job
    d. 4.D Gestione della Richiesta: Funzione process_job
5. **Considerazioni Finali**

1 Introduzione

Il progetto sviluppato consiste in un sistema client-server per il calcolo dell'hash crittografico SHA-256 di file arbitrari, realizzato interamente in linguaggio C come esercizio applicativo per il corso di Sistemi Operativi. La comunicazione tra client e server si basa su FIFO (named pipe), secondo il modello POSIX, e avviene tramite una FIFO pubblica condivisa per l'invio delle richieste e FIFO temporanee private per la ricezione dei risultati.

Il client ha il compito di inviare al server, per ciascun file specificato, una richiesta contenente il percorso del file e il nome della FIFO privata su cui attende il risultato. Dopo aver ricevuto gli hash SHA-256 calcolati, li ordina e visualizza in base alla dimensione dei file elaborati.

Il server, avviato in parallelo, riceve continuamente richieste tramite la FIFO pubblica. Ogni richiesta viene inserita in una coda ordinata per dimensione del file e assegnata a un thread worker, rispettando un limite massimo di thread attivi configurabile. Il calcolo dell'hash avviene tramite la libreria OpenSSL, ed è ottimizzato grazie a un sistema di caching interno: se un file è già stato elaborato, il risultato viene restituito immediatamente, evitando ricalcoli. Inoltre, se più thread richiedono lo stesso file simultaneamente, il server garantisce che solo uno esegua l'elaborazione, mentre gli altri attendono il risultato già in corso di calcolo.

Nel complesso, il sistema è stato progettato per gestire in modo efficiente la concorrenza, la sincronizzazione e l'organizzazione delle richieste, offrendo un'infrastruttura robusta per il calcolo distribuito di hash in ambiente multi-processo.

2 Struttura Generale del Sistema

Il sistema realizzato è costituito da due programmi principali: un **client** (`the_client.c`) e un **server** (`the_server.c`). Il client si occupa di generare e inviare richieste per il calcolo dell'hash SHA-256 di file indicati dall'utente; il server riceve tali richieste tramite FIFO e le elabora in modo concorrente, sfruttando thread multipli, una cache condivisa e un meccanismo di ordinamento delle richieste per dimensione del file.

2.1 Il Client – `the_client.c`

Il programma client gestisce l'interazione iniziale con l'utente e prepara le richieste per ciascun file indicato come argomento da riga di comando. Per ogni file:

- Determina il percorso assoluto e la dimensione del file.
- Crea una FIFO di risposta temporanea, con nome univoco basato su `getpid()` e un indice numerico (es. `/tmp/fifo_response_<pid>_<n>`).
- Invia al server una richiesta formattata come `filepath::fifo_client`, scrivendola nella FIFO pubblica `/tmp/fifo_in`.

Dopo aver inviato tutte le richieste, il client apre in lettura le FIFO di risposta una alla volta, attendendo l'hash SHA-256 restituito dal server. L'hash viene letto come una stringa esadecimale di 64 caratteri. Se la lettura fallisce, viene inserito un messaggio di errore.

Terminata la raccolta di tutti gli hash, il client effettua un **ordinamento interno** dei risultati in base alla dimensione dei file (ordine crescente). I risultati finali vengono infine stampati a schermo, includendo il percorso del file, la sua dimensione in byte e l'hash corrispondente.

2.2 Il Server – `the_server.c`

Il programma server è progettato per **gestire più richieste in parallelo** tramite l'uso di **thread** e **FIFO**. All'avvio, crea la FIFO pubblica `/tmp/fifo_in` per ricevere richieste da uno o più client. Un **thread dispatcher dedicato** legge continuamente dalla FIFO e inserisce ogni richiesta in una **coda interna ordinata** per dimensione del file, con priorità ai file più piccoli.

Le richieste vengono gestite da thread worker creati dinamicamente, fino a un massimo di thread attivi (ad esempio 4), definito da una costante (`THREAD_LIMIT`). Se tutti i thread sono occupati, le richieste restano in coda fino a quando una risorsa non diventa disponibile.

Il server implementa inoltre due meccanismi fondamentali per l'efficienza:

- **Caching dei risultati**: ogni hash calcolato viene memorizzato in una cache in memoria. Se una richiesta riguarda un file già processato, l'hash può essere restituito immediatamente.
- **Gestione delle richieste in corso**: se più thread ricevono contemporaneamente una richiesta per lo stesso file, uno solo esegue il calcolo, mentre gli altri attendono la fine dell'elaborazione tramite `pthread_cond_wait`. Questo evita ricalcoli ridondanti e migliora l'uso delle risorse.

Una volta ottenuto l'hash SHA-256 tramite OpenSSL, il thread worker scrive il risultato sulla FIFO indicata nella richiesta (creata dal client). Infine, la FIFO viene chiusa e il thread si dichiara disponibile per nuove elaborazioni.

Questa architettura garantisce la **scalabilità** del sistema, una buona **responsività** per i file più piccoli, e un **riuso efficiente** dei risultati grazie alla cache condivisa.

3. Analisi Tecnologica del Sistema

3.A Meccanismi FIFO e Generazione Dinamica dei Canali di Risposta

La comunicazione tra client e server è basata sull'uso di FIFO (named pipe) secondo la semantica POSIX. All'avvio, il server crea una FIFO pubblica, `/tmp/fifo_in`, che funge da canale unico per ricevere tutte le richieste dei client.

Ogni client, per ricevere la risposta dal server, crea dinamicamente una FIFO privata con un nome univoco, ad esempio `/tmp/fifo_response_<PID>_<N>`, dove `<PID>` è il Process ID del client e `<N>` un indice incrementale. Questa FIFO privata viene creata tramite la chiamata `mkfifo()` e il suo nome viene inviato insieme al percorso del file da processare nel messaggio inviato al server, usando una stringa nel formato `filepath::fifo_path`.

In questo modo, ogni richiesta è associata a un canale di risposta esclusivo, evitando conflitti e sovrapposizioni tra client concorrenti. Il server, dopo aver calcolato l'hash, apre la FIFO privata in scrittura e invia il risultato. Il client, una volta ricevuto il valore, legge dalla FIFO e la elimina con `unlink()` per liberare risorse.

Per evitare che il server si blocchi in attesa di scrittori sulla FIFO pubblica (ad esempio se nessun client è connesso), questa viene aperta in modalità `O_RDWR`. Questo accorgimento garantisce che il server resti sempre in ascolto e la comunicazione sia robusta e non soggetta a deadlock.

3.B Calcolo dell'Hash SHA-256 con la Libreria OpenSSL

Il server esegue il calcolo dell'impronta SHA-256 utilizzando la libreria OpenSSL, nota per l'efficienza e la sicurezza crittografica. Il processo avviene tramite tre funzioni principali:

- `SHA256_Init()` per inizializzare il contesto di hashing;
- `SHA256_Update()` chiamata ripetutamente per leggere blocchi di dati dal file (tipicamente 1024 byte per volta) e aggiornare il contesto;
- `SHA256_Final()` per ottenere il digest finale di 32 byte.

Ogni thread worker, incaricato di processare una richiesta, esegue questa procedura sul file indicato. Successivamente, il digest binario viene convertito in una stringa esadecimale di 64 caratteri tramite `sprintf`, rendendo leggibile e trasferibile l'hash.

L'uso di OpenSSL elimina la necessità di implementare manualmente algoritmi crittografici complessi, garantendo standard di sicurezza elevati e prestazioni ottimali.

3.C Politica di Priorità Basata sulla Dimensione dei File

Diversamente da una semplice gestione FIFO, il server adotta una politica di priorità per processare prima i file di dimensione minore. La funzione `enqueue_job()` inserisce ogni nuova richiesta nella coda mantenendo un ordinamento crescente per dimensione del file.

Se due file hanno dimensione identica, viene utilizzato un criterio secondario basato sull'ordinamento lessicografico della stringa di richiesta per determinare la precedenza.

Questa strategia consente di migliorare il tempo medio di risposta del sistema, privilegiando i file più piccoli che possono essere elaborati più velocemente,

specialmente in situazioni di elevato carico. Sebbene ciò si discosti dal comportamento FIFO puro, è una scelta motivata da esigenze pratiche di efficienza e reattività.

3.D Architettura Concorrente: Thread, Blocchi Critici e Sincronizzazione

Per gestire più richieste simultanee, il server utilizza un modello multithreading con un numero massimo di thread attivi, controllato tramite la variabile condivisa `running_threads`.

Questa variabile è protetta da un mutex (`thread_lock`) e associata a una variabile di condizione (`pthread_cond_t`). Se il numero di thread raggiunge il limite massimo (ad esempio 4), il dispatcher si sospende in attesa che un thread completi il proprio lavoro e liberi uno slot.

Altre strutture condivise includono:

- La coda delle richieste, protetta da `queue_lock`;
- La cache degli hash e la lista delle elaborazioni in corso, protette da `cache_lock`.

L'uso di mutex e variabili di condizione evita situazioni di busy-waiting e garantisce che le operazioni critiche su dati condivisi siano eseguite in modo atomico e sincronizzato, prevenendo race condition e inconsistenze.

3.E Strategia di Memorizzazione dei Risultati e Coordinamento delle Elaborazioni

Per aumentare l'efficienza e ridurre i tempi di elaborazione, il server mantiene una cache (`cache[]`) con i file già processati e i rispettivi hash.

Prima di eseguire il calcolo, il server verifica se il file è presente in cache: in tal caso restituisce immediatamente l'hash, evitando un ricalcolo inutile. L'accesso a questa struttura è sincronizzato tramite `cache_lock` per assicurare consistenza in ambiente multithread.

Inoltre, per evitare che più thread elaborino contemporaneamente lo stesso file, il server usa una lista `active_list[]` che traccia i file in elaborazione.

Se una nuova richiesta arriva mentre il file è già in elaborazione, il thread associato si mette in attesa su una `pthread_cond_t` specifica per quella entry.

Quando il calcolo termina, il thread che ha finito aggiorna la cache e risveglia tutti i thread in attesa con `pthread_cond_broadcast`.

Questa strategia di coordinamento evita ridondanze computazionali e consente di gestire richieste simultanee in modo efficiente, particolarmente utile quando diversi client richiedono hash per lo stesso file.

4 Esame di Sezioni Chiave del Codice Sorgente

Per illustrare meglio l'architettura e il funzionamento del progetto, in questa sezione analizziamo frammenti significativi del codice implementato in `the_client.c` e `the_server.c`, soffermandoci su alcune componenti cruciali del sistema.

4.A Costruzione delle Richieste nel Lato Client

All'interno del file `the_client.c`, la funzione `main` è responsabile della preparazione delle richieste. Per ciascun file indicato, il client:

- raccoglie il percorso e la dimensione tramite `stat`
- genera dinamicamente una FIFO temporanea di risposta (basata su `getpid()`)
- crea la FIFO con `mkfifo`
- scrive una stringa formattata `path::fifo` sulla FIFO comune `/tmp/fifo_in`

Tale stringa sarà ricevuta dal server, che successivamente invierà l'hash sulla FIFO indicata. Dopo aver ricevuto tutte le risposte, il client elimina ogni FIFO creata.

4.B Algoritmo di Hash: Funzione `compute_sha256`

Nel file `the_server.c`, la funzione `compute_sha256` incapsula l'intero processo di calcolo SHA-256. Essa:

- inizializza un contesto OpenSSL tramite `SHA256_Init`
- legge il contenuto del file in blocchi (1024 byte) con `read`
- aggiorna il contesto con `SHA256_Update`
- chiude il file e conclude con `SHA256_Final`, restituendo l'hash binario

Il risultato viene poi convertito in stringa esadecimale tramite `sprintf` e inviato al client.

4.C Inserimento Ordinato: Funzione `enqueue_job`

Il server organizza la coda di richieste secondo la dimensione dei file. La funzione `enqueue_job`:

- acquisisce un mutex per proteggere la coda
- effettua uno spostamento degli elementi per mantenere l'ordinamento crescente
- gestisce casi di parità con confronto alfabetico
- invia una `pthread_cond_signal` per notificare la disponibilità della coda

Questo garantisce che file piccoli vengano processati prima, migliorando la reattività.

4.D Gestione della Richiesta: Funzione `process_job`

Ogni thread worker esegue la funzione `process_job`. Essa esegue:

- il parsing della stringa di richiesta (`filepath::fifo_path`)
- la verifica in cache
- l'accesso alla lista `active_list` per verificare se un altro thread sta già elaborando lo stesso file
- l'attesa tramite `pthread_cond_wait` in caso di elaborazione in corso
- il calcolo dell'hash solo se necessario
- la scrittura nella FIFO privata del client
- l'aggiornamento dello stato condiviso e la pulizia delle risorse

Grazie all'utilizzo di `mutex`, `cond`, e `detach`, la funzione mantiene l'efficienza del server anche in condizioni di elevato carico.

Questi frammenti dimostrano come la nostra implementazione affronti con precisione tematiche di sincronizzazione, concorrenza e comunicazione tra processi.

5 Considerazioni Finali

Il sistema sviluppato rappresenta un'applicazione completa ed efficace dei concetti fondamentali di programmazione concorrente, comunicazione tra processi e ottimizzazione delle risorse in ambiente Unix/Linux. Attraverso un'architettura basata su FIFO, multithreading controllato e caching intelligente, il progetto realizza un servizio robusto per il calcolo distribuito di impronte SHA-256 su file multipli.

Dal punto di vista prestazionale, l'uso di un numero massimo di thread worker consente di sfruttare in modo equilibrato le risorse multicore, evitando saturazioni e garantendo la stabilità del server anche in presenza di numerose richieste. L'ordinamento delle richieste per dimensione dei file e la gestione centralizzata tramite coda permettono di ottimizzare i tempi di risposta, privilegiando elaborazioni brevi e migliorando la reattività generale del sistema.

Un altro punto di forza risiede nella gestione efficiente delle duplicazioni: grazie al meccanismo di caching e alla sincronizzazione tra thread su file condivisi, il sistema evita calcoli ridondanti e gestisce correttamente richieste concorrenti per lo stesso file. Ciò contribuisce in modo significativo a ridurre il carico computazionale e a migliorare l'efficienza complessiva.

Dal punto di vista della scalabilità, il sistema si è dimostrato in grado di gestire simultaneamente più client, mantenendo buone prestazioni e correttezza nella comunicazione. I parametri come `MAX_THREADS`, dimensione della cache e capacità della coda possono essere facilmente adattati in base alle caratteristiche dell'hardware e al carico previsto.

In conclusione, il progetto soddisfa pienamente i requisiti dell'opzione 2 del corso di Sistemi Operativi, realizzando un'infrastruttura solida, modulare ed estensibile. Le scelte architetturali adottate permettono un uso efficiente delle risorse e pongono le basi per eventuali estensioni future, come la sostituzione FIFO della cache, il supporto a una rete distribuita o l'introduzione di una GUI

client-side. Già nella versione attuale, il sistema è pronto per scenari d’uso realistici in ambiente monomacchina.